E4 Educator v2.0

Tier 3 · Project 4 of 5

P04

BANDSCOPE Audio Analyser

Walark Electronics · Fundrum Engineering · May 2026

Project overview

BANDSCOPE is a complete real-time audio spectrum analyser and acoustic event detector built on the E4's ICS43434 MEMS microphone and I2S peripheral. It delivers a 60-bar FFT spectrum display at ~20fps, multi-mode operation (spectrum, octave band, volume meter, event log), MQTT telemetry of acoustic events and ambient noise levels, SD logging of sound levels, and a configurable event detection engine. No external audio hardware is required.

New beyond T16 BANDSCOPE sketch:

Four display modes · Octave band analyser · Configurable event detection engine · Event log with timestamps · MQTT noise level and event telemetry · SD dB level logging · Core 1 task for audio processing · Gain and sensitivity controls · Web dashboard with live level graph

1 Project specification

1.1 Feature specification

ID	Feature	Detail
F01	Spectrum analyser	60-bar FFT display. 16kHz sample rate. 512-point FFT. ~20fps. Peak hold with 1.5s decay. Smoothing.
F02	Octave band display	10 standard ISO octave bands: 31Hz to 16kHz. Each band shows averaged energy. Same colour scheme.
F03	Volume meter mode	Full-width horizontal RMS bar with dB SPL readout. Slow/fast ballistics. Over indicator.
F04	Event log mode	Scrolling list of last 10 detected events with timestamp (millis or RTC if fitted) and type.
F05	Event detection engine	Configurable detectors: clap, tone (4 configurable frequencies), broadband threshold, sustained noise.
F06	Gain control	Software gain multiplier 1x-16x. Adjusted via NEXT long press + ENT. Stored in NVS.
F07	MQTT telemetry	e4/audio/dBSPL published every 5s (retained). e4/audio/event published on each detection.
F08	SD level logging	dB SPL logged to AUDIO_LOG.CSV every 10s. Includes peak and average per interval.
F09	Core 1 audio task	FFT computation runs on Core 1 (Arduino uses Core 1 by default). I2S reads on Core 0 via task.
F10	Web dashboard	Live dB level bar and chart. Event log. Gain control slider. Mode selector.
F11	Touch controls	Footer: MODE · GAIN+ · HOLD. Physical NEXT=mode cycle, ENT=peak reset/gain adjust.
F12	OTA updates	ElegantOTA at /update. Audio processing paused during update.

1.2 Display modes

Mode	Header label	Content
MODE_SPECTRUM	BANDSCOPE • SPECTRUM	60-bar FFT. Green/orange/red by frequency. Peak hold dots. Dominant freq in header.
MODE_OCTAVE	BANDSCOPE • OCTAVE	10 ISO octave band bars. Band centre frequency labels. Log-scaled.
MODE_VOLUME	BANDSCOPE • LEVEL	Large dB SPL value. Horizontal bar. Green/yellow/red zones. Slow ballistics.
MODE_EVENTS	BANDSCOPE • EVENTS	Scrolling event log. 10 most recent events. Type, level, time.

2 Architecture — dual-core audio processing

The ESP32 has two Xtensa LX6 cores. By default, the Arduino framework runs your code on Core 1. For BANDSCOPE, audio capture and FFT computation run continuously and must not be starved by WiFi activity. The solution is to use a FreeRTOS task pinned to Core 0 for audio processing while Core 1 handles the Arduino loop, display, WiFi, and MQTT.

2.1 Core assignment

Core	Responsibilities	Notes
Core 0	I2S audio capture · FFT computation · Magnitude calculation · Bar values · Event detection	FreeRTOS task pinned to core 0. Highest priority. 4KB stack.
Core 1	TFT sprite rendering · WiFi/MQTT · Web server · ElegantOTA · Button/touch · SD logging	Arduino loop() runs here. Reads shared audio data via mutex.

Key point

Running I2S capture and FFT on Core 0 prevents WiFi interrupt handler activity on Core 1 from causing I2S DMA buffer overruns. Without this separation, WiFi activity causes audible gaps in the capture and irregular FFT frames. The dual-core approach is the production-quality solution.

2.2 Shared data structure and mutex

```
#include <Arduino.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/semphr.h>

#define FFT_SIZE     512
#define NUM_BARS      60
#define FIRST_BIN      2
#define SAMPLE_RATE 16000

// ── Shared audio data (written by Core 0, read by Core 1) ────
struct AudioData {
  float   bars[NUM_BARS];       // Smoothed bar heights 0-240
  int     peakHold[NUM_BARS];   // Peak hold heights
  unsigned long peakTime[NUM_BARS];
  float   dBSPL;                // Current RMS level in dB SPL
  float   peakDBSPL;            // Peak dB SPL in current interval
  float   dominantFreq;         // FFT peak frequency Hz
  bool    eventClap;            // Set true on clap detection
  uint8_t eventTone;            // Tone frequency bin if detected
  bool    newFrame;             // True when Core 0 wrote new data
};

AudioData audioData;
SemaphoreHandle_t audioMutex;

// Safely read audio data from Core 1
bool getAudioData(AudioData& out) {
```

```
  if (xSemaphoreTake(audioMutex, pdMS_TO_TICKS(10)) == pdTRUE) {
    out = audioData;
    audioData.newFrame = false;
    audioData.eventClap = false;
    audioData.eventTone = 0;
    xSemaphoreGive(audioMutex);
    return true;
  }
  return false;
}

// Safely write from Core 0
void setAudioData(const AudioData& in) {
  if (xSemaphoreTake(audioMutex, pdMS_TO_TICKS(5)) == pdTRUE) {
    audioData = in;
    xSemaphoreGive(audioMutex);
  }
}
```

3 Audio processing task — Core 0

```
#include <driver/i2s.h>
#include <arduinoFFT.h>
#include <math.h>

double  vReal[FFT_SIZE];
double  vImag[FFT_SIZE];
int32_t i2sBuf[FFT_SIZE];

arduinoFFT fft(vReal, vImag, FFT_SIZE, SAMPLE_RATE);

#define PEAK_HOLD_MS  1500
#define SMOOTH_FACTOR 0.30f
#define CLAP_THRESH   0.12f
#define CLAP_COOL_MS   500

// Gain multiplier — set from Core 1 (NVS loaded)
volatile float gainMultiplier = 1.0f;

// Local audio state (Core 0 only)
float prevBars[NUM_BARS]   = {};
int   localPeak[NUM_BARS]  = {};
unsigned long localPeakT[NUM_BARS] = {};
unsigned long lastClapT    = 0;

void setupI2S() {
  i2s_config_t cfg = {
    .mode             = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
    .sample_rate      = SAMPLE_RATE,
    .bits_per_sample  = I2S_BITS_PER_SAMPLE_32BIT,
    .channel_format   = I2S_CHANNEL_FMT_ONLY_LEFT,
    .communication_format = I2S_COMM_FORMAT_STAND_I2S,
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
    .dma_buf_count    = 4,
    .dma_buf_len      = FFT_SIZE / 2,
    .use_apll         = false,
    .tx_desc_auto_clear = false,
    .fixed_mclk       = 0
  };
  i2s_pin_config_t pins = {
    .bck_io_num   = MEMS_BCLK,
    .ws_io_num    = MEMS_WS,
    .data_out_num = I2S_PIN_NO_CHANGE,
    .data_in_num  = MEMS_SD
  };
  i2s_driver_install(I2S_NUM_0, &cfg, 0, nullptr);
  i2s_set_pin(I2S_NUM_0, &pins);
  i2s_zero_dma_buffer(I2S_NUM_0);
}

float rmsToDbSPL(float rms) {
  if (rms < 1.0f) return 0.0f;
  float dbFS = 20.0f * log10f(rms / 8388608.0f);
```

```
  return dbFS + 94.0f + 26.0f;
}

void audioTask(void* param) {
  setupI2S();
  AudioData local = {};
  unsigned long now;

  for (;;) {
    // ── Read I2S samples ──────────────────────────────
    size_t bytesRead = 0;
    i2s_read(I2S_NUM_0, i2sBuf,
             FFT_SIZE * sizeof(int32_t),
             &bytesRead, portMAX_DELAY);
    size_t n = bytesRead / sizeof(int32_t);

    // ── Build FFT input with gain ─────────────────────
    double rmsSum = 0;
    for (size_t i = 0; i < FFT_SIZE; i++) {
      double s = (i < n)
        ? (double)(i2sBuf[i] >> 8) / 8388608.0 * gainMultiplier
        : 0.0;
      vReal[i] = s;
      vImag[i] = 0.0;
      rmsSum += s * s;
    }
    float rms = sqrtf((float)(rmsSum / FFT_SIZE));

    // ── FFT ──────────────────────────────────────────
    fft.windowing(FFT_WIN_TYP_HANN, FFT_FORWARD);
    fft.compute(FFT_FORWARD);
    fft.complexToMagnitude();

    now = millis();

    // ── Map bins to bars ──────────────────────────────
    for (int b = 0; b < NUM_BARS; b++) {
      int bin = FIRST_BIN + b * 2;
      if (bin + 1 >= FFT_SIZE / 2) break;
      double mag = (vReal[bin] + vReal[bin+1]) * 0.5;
      float dB = (mag > 0.0)
                 ? 20.0f * log10f((float)mag)
                 : -80.0f;
      float h = map((long)constrain((int)dB, -80, -10),
                    -80, -10, 0, 240);
      local.bars[b] = prevBars[b] * SMOOTH_FACTOR
                    + h * (1.0f - SMOOTH_FACTOR);
      prevBars[b] = local.bars[b];

      // Peak hold
      int barH = (int)local.bars[b];
      if (barH >= localPeak[b]) {
        localPeak[b] = barH; localPeakT[b] = now;
      } else if (now - localPeakT[b] > PEAK_HOLD_MS) {
```

```
        if (localPeak[b] > 0) localPeak[b]--;
      }
      local.peakHold[b] = localPeak[b];
      local.peakTime[b] = localPeakT[b];
    }

    // ── Level ──────────────────────────────────────────
    local.dBSPL        = rmsToDbSPL(rms * 8388608.0f);
    if (local.dBSPL > local.peakDBSPL)
      local.peakDBSPL = local.dBSPL;
    local.dominantFreq = (float)fft.majorPeak();

    // ── Clap detection ─────────────────────────────────
    float broadband = 0;
    for (int i = 2; i < 200 && i < FFT_SIZE/2; i++)
      broadband += vReal[i];
    broadband /= 198.0f;
    if (broadband > CLAP_THRESH
        && now - lastClapT > CLAP_COOL_MS) {
      local.eventClap = true;
      lastClapT = now;
    }

    local.newFrame = true;
    setAudioData(local);
    local.eventClap  = false;
    local.eventTone  = 0;
    local.peakDBSPL  = local.dBSPL;  // Reset peak each frame
    taskYIELD();
  }
}
```

4 Display modules

4.1 Spectrum display

```
#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_FOOTER  0x18C3
#define COL_TEXT    0xFFFF
#define COL_LABEL   0xC618
#define COL_GRID    0x1082
#define COL_BAR_LO  0x07E0   // Green
#define COL_BAR_MID 0xFD20   // Orange
#define COL_BAR_HI  0xF800   // Red
#define COL_PEAK    0xFFFF   // White peak dots

#define SPEC_X    0
#define SPEC_Y   42
#define SPEC_W  240
#define SPEC_H  240
#define BAR_W     3
#define BAR_GAP   1

void renderSpectrum(const AudioData& ad) {
  spr.fillSprite(COL_BG);

  // Horizontal grid lines at -10/-20/-40/-60 dB equivalent
  for (int g = 40; g < SPEC_H; g += 40)
    spr.drawFastHLine(0, SPEC_H - g, SPEC_W, COL_GRID);

  for (int i = 0; i < NUM_BARS; i++) {
    int barH = constrain((int)ad.bars[i], 0, SPEC_H);
    int x    = i * (BAR_W + BAR_GAP);
    int y    = SPEC_H - barH;

    uint16_t col = (i < 15) ? COL_BAR_LO :
                   (i < 40) ? COL_BAR_MID : COL_BAR_HI;
    if (barH > 0) spr.fillRect(x, y, BAR_W, barH, col);

    // Peak hold dot
    int ph = constrain(ad.peakHold[i], 0, SPEC_H);
    if (ph > 0)
      spr.fillRect(x, SPEC_H - ph, BAR_W, 2, COL_PEAK);
  }
  spr.pushSprite(SPEC_X, SPEC_Y);

  // Dominant frequency in header
  tft.fillRect(120, 12, 112, 16, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(1);
  tft.setCursor(122, 16);
  tft.printf("%.0f Hz  %.1fdB",
             ad.dominantFreq, ad.dBSPL);
}
```

4.2 Octave band display

The octave band display maps the FFT bins to the 10 standard ISO octave band centre frequencies. Each band averages the energy of the FFT bins within its frequency range:

```
// ISO octave band centre frequencies (Hz)
const float OCTAVE_CENTRES[] = {
  31.5f, 63.0f, 125.0f, 250.0f, 500.0f,
  1000.0f, 2000.0f, 4000.0f, 8000.0f, 16000.0f
};
const char* OCTAVE_LABELS[] = {
  "31","63","125","250","500","1k","2k","4k","8k","16k"
};
#define NUM_OCTAVES 10

float octaveBars[NUM_OCTAVES] = {};
float prevOctaveBars[NUM_OCTAVES] = {};

void computeOctaveBands() {
  float binHz = (float)SAMPLE_RATE / FFT_SIZE;

  for (int band = 0; band < NUM_OCTAVES - 1; band++) {
    // Band spans half-octave either side of centre
    float fLow  = OCTAVE_CENTRES[band]   / 1.414f;
    float fHigh = OCTAVE_CENTRES[band]   * 1.414f;
    int   binLo = (int)(fLow  / binHz);
    int   binHi = (int)(fHigh / binHz);
    binLo = max(binLo, 1);
    binHi = min(binHi, FFT_SIZE/2 - 1);

    double sum = 0;
    int count = 0;
    for (int b = binLo; b <= binHi; b++) {
      sum += vReal[b]; count++;
    }
    if (count == 0) { octaveBars[band] = 0; continue; }
    float avg = (float)(sum / count);
    float dB  = (avg > 0) ? 20.0f * log10f(avg) : -80.0f;
    float h   = map((long)constrain((int)dB, -80, -10),
                    -80, -10, 0, 220);
    octaveBars[band] = prevOctaveBars[band] * 0.4f
                     + h * 0.6f;
    prevOctaveBars[band] = octaveBars[band];
  }
}

void renderOctaveBands() {
  spr.fillSprite(COL_BG);
  int barW = 22; int gap = 2;
  for (int i = 0; i < NUM_OCTAVES; i++) {
    int x    = 2 + i * (barW + gap);
    int barH = constrain((int)octaveBars[i], 0, 220);
    int y    = 220 - barH;
    uint16_t col = (i < 3) ? COL_BAR_LO :
```

```
                    (i < 7) ? COL_BAR_MID : COL_BAR_HI;
    if (barH > 0) spr.fillRect(x, y, barW, barH, col);
    spr.drawRect(x, 0, barW, 220, COL_GRID);
    spr.setTextColor(COL_LABEL, COL_BG);
    spr.setTextSize(1);
    spr.setCursor(x + 2, 224);
    spr.print(OCTAVE_LABELS[i]);
  }
  spr.pushSprite(0, 42);
}
```

4.3 Volume meter mode

```
float slowLevel = 0.0f;  // Slow ballistics for VU-style display

void renderVolumeMeter(const AudioData& ad) {
  // Slow ballistics: 300ms attack, 600ms decay
  if (ad.dBSPL > slowLevel)
    slowLevel = slowLevel * 0.3f + ad.dBSPL * 0.7f;  // Fast attack
  else
    slowLevel = slowLevel * 0.8f + ad.dBSPL * 0.2f;  // Slow decay

  float displayDb = constrain(slowLevel, 30.0f, 90.0f);
  int barW = (int)map((long)displayDb, 30, 90, 0, 220);

  spr.fillSprite(COL_BG);

  // Big dB readout
  spr.setTextColor(COL_TEXT, COL_BG);
  spr.setTextSize(5);
  spr.setCursor(8, 30);
  spr.printf("%.0f", ad.dBSPL);
  spr.setTextSize(2);
  spr.setCursor(8, 96);
  spr.print("dB SPL");

  // Horizontal level bar with colour zones
  spr.drawRect(10, 130, 220, 30, COL_LABEL);
  // Green zone: 30-60 dB
  int w1 = min(barW, 110);
  if (w1 > 0) spr.fillRect(11, 131, w1, 28, COL_BAR_LO);
  // Yellow zone: 60-75 dB
  if (barW > 110) {
    int w2 = min(barW - 110, 55);
    spr.fillRect(121, 131, w2, 28, COL_WARN);
  }
  // Red zone: 75-90 dB
  if (barW > 165) {
    spr.fillRect(176, 131, barW - 165, 28, COL_BAR_HI);
  }

  // Scale marks
  spr.setTextColor(COL_LABEL, COL_BG);
```

```
  spr.setTextSize(1);
  spr.setCursor(8,  168); spr.print("30");
  spr.setCursor(108, 168); spr.print("60");
  spr.setCursor(163, 168); spr.print("75");
  spr.setCursor(218, 168); spr.print("90");

  // Gain indicator
  spr.setCursor(8, 190);
  spr.printf("Gain: x%.0f", gainMultiplier);

  spr.pushSprite(0, 42);
}
```

5 Configurable event detection engine

The event detection engine runs inside the audio task on Core 0. It supports multiple detector types that can be individually enabled and configured via NVS. Detected events are stored in a shared event queue that Core 1 reads for display, MQTT, and logging.

5.1 Detector types

Detector	Trigger condition	Configuration
CLAP	Broadband average exceeds threshold in single frame	clapThresh (default 0.12), cooldownMs (500)
TONE_A/B/C/D	Energy in target bin significantly above neighbours	toneFreqA/B/C/D (Hz), toneSensitivity (default 3.0x)
LOUD	dB SPL exceeds threshold	loudThreshDB (default 75), sustainedMs (500)
QUIET	dB SPL falls below threshold	quietThreshDB (default 35), sustainedMs (2000)

5.2 Event queue and display

```
#define MAX_EVENTS 10

struct AudioEvent {
  unsigned long timestampMs;
  char          type[12];   // "CLAP", "TONE_1kHz", "LOUD", "QUIET"
  float         levelDB;
};

AudioEvent eventLog[MAX_EVENTS];
int         eventCount = 0;
SemaphoreHandle_t eventMutex;

void addEvent(const char* type, float levelDB) {
  if (xSemaphoreTake(eventMutex, pdMS_TO_TICKS(5)) != pdTRUE) return;
  // Shift existing events down
  if (eventCount < MAX_EVENTS) eventCount++;
  for (int i = eventCount - 1; i > 0; i--)
    eventLog[i] = eventLog[i-1];
  // Insert new event at front
  eventLog[0].timestampMs = millis();
  strncpy(eventLog[0].type, type, 11);
  eventLog[0].levelDB = levelDB;
  xSemaphoreGive(eventMutex);

  Serial.printf("EVENT: %s %.1fdB\n", type, levelDB);
}

void renderEventLog() {
  spr.fillSprite(COL_BG);
  if (xSemaphoreTake(eventMutex, pdMS_TO_TICKS(10)) != pdTRUE) {
    spr.pushSprite(0, 42); return;
```

```
  }
  for (int i = 0; i < eventCount && i < MAX_EVENTS; i++) {
    int y = i * 22;
    uint16_t col = (strncmp(eventLog[i].type,"LOUD",4)==0) ? COL_ERR :
                   (strncmp(eventLog[i].type,"CLAP",4)==0) ? COL_WARN :
                   COL_OK;
    spr.setTextColor(col, COL_BG);
    spr.setTextSize(1);
    spr.setCursor(4, y + 4);
    spr.printf("%-10s %5.1fdB  %lus ago",
               eventLog[i].type,
               eventLog[i].levelDB,
               (millis() - eventLog[i].timestampMs) / 1000);
  }
  xSemaphoreGive(eventMutex);
  spr.pushSprite(0, 42);
}
```

6 MQTT telemetry and SD logging

```
// MQTT topic structure for BANDSCOPE
// e4/audio/dBSPL        ← ambient level, published every 5s, retained
// e4/audio/event        ← event type+level JSON, on detection
// e4/audio/peak         ← peak dB in last 5s window, retained
// e4/audio/dominantFreq ← dominant frequency Hz, published every 5s

unsigned long lastMQTT = 0;
unsigned long lastLog  = 0;
float intervalPeakDB   = 0;

void publishAudio(const AudioData& ad) {
  if (!mqtt.connected()) return;

  unsigned long now = millis();

  // Track peak over MQTT interval
  if (ad.dBSPL > intervalPeakDB) intervalPeakDB = ad.dBSPL;

  // Publish level + dominant freq every 5 seconds
  if (now - lastMQTT >= 5000) {
    lastMQTT = now;
    char buf[16];
    snprintf(buf, sizeof(buf), "%.1f", ad.dBSPL);
    mqtt.publish("e4/audio/dBSPL", buf, true);
    snprintf(buf, sizeof(buf), "%.1f", intervalPeakDB);
    mqtt.publish("e4/audio/peak", buf, true);
    snprintf(buf, sizeof(buf), "%.0f", ad.dominantFreq);
    mqtt.publish("e4/audio/dominantFreq", buf, false);
    intervalPeakDB = 0;
  }

  // Publish events immediately
  if (ad.eventClap) {
    char json[48];
    snprintf(json, sizeof(json),
             "{\"type\":\"CLAP\",\"dB\":%.1f}",
             ad.dBSPL);
    mqtt.publish("e4/audio/event", json, false);
  }
}

// SD: log average + peak dB every 10 seconds
#define SD_LOG_FILE "/AUDIO_LOG.CSV"

void logAudioLevel(const AudioData& ad) {
  if (!sdOk) return;
  unsigned long now = millis();
  if (now - lastLog < 10000) return;
  lastLog = now;

  static float sumDB = 0; static int count = 0;
```

```
  static float peakDB = 0;
  sumDB  += ad.dBSPL;
  count++;
  if (ad.dBSPL > peakDB) peakDB = ad.dBSPL;

  char line[48];
  snprintf(line, sizeof(line), "%lu,%.1f,%.1f",
           now, sumDB / count, peakDB);
  File f = SD.open(SD_LOG_FILE, FILE_APPEND);
  if (f) { f.println(line); f.close(); }

  sumDB = 0; count = 0; peakDB = 0;
}
```

7 main.cpp — coordinator

```
#include <Arduino.h>
#include <esp_ota_ops.h>
#include "AudioTask.h"
#include "Display.h"
#include "Network.h"
#include "Logger.h"
#include "Settings.h"
#include "Button.h"

TFT_eSPI    tft;
TFT_eSprite spr = TFT_eSprite(&tft);
Button      nextBtn(BTN_NEXT);
Button      entBtn(BTN_ENT);
SettingsManager settings;

enum Mode { MODE_SPECTRUM, MODE_OCTAVE, MODE_VOLUME, MODE_EVENTS };
Mode currentMode = MODE_SPECTRUM;

bool otaValid    = false;
AudioData latestAudio;

void setup() {
  Serial.begin(115200);
  Serial.printf("P04 BANDSCOPE  FW:%s\n", FW_VERSION);

  settings.load();
  gainMultiplier = settings.s.audioGain;

  // NOTE: I2S uses GPIO 32 (MEMS_BCLK) = LED_BLUE pin
  // Do NOT init or use LED_BLUE in BANDSCOPE firmware
  pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
  pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);

  // TFT init
  pinMode(TFT_BL, OUTPUT); digitalWrite(TFT_BL, LOW);
  tft.init(); tft.setRotation(0); tft.fillScreen(0x0000);  // Landscape 320x240
  ledcSetup(0, 5000, 8); ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, settings.s.brightness);

  spr.setColorDepth(16);
  spr.createSprite(320, 165);  // Content zone

  nextBtn.begin(); entBtn.begin();

  // Mutexes
  audioMutex = xSemaphoreCreateMutex();
  eventMutex = xSemaphoreCreateMutex();

  // SD
  sdOk = SD.begin(SD_CS);  // After TFT init
  if (sdOk) initAudioLog();
```

```
  // Network (async)
  initNetwork();

  // Draw static chrome
  drawChrome("BANDSCOPE • SPECTRUM");
  drawFooter();

  // Launch audio task on Core 0, priority 2, 4KB stack
  xTaskCreatePinnedToCore(
    audioTask,       // Task function
    "AudioTask",     // Name
    8192,            // Stack size bytes
    nullptr,         // Parameter
    2,               // Priority
    nullptr,         // Task handle
    0);              // Core 0

  digitalWrite(LED_GREEN, HIGH);
  Serial.println("BANDSCOPE running. Audio on Core 0.");
}

void loop() {
  unsigned long now = millis();

  wifiUpdate(now);
  mqttUpdate(now);
  ElegantOTA.loop();

  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();
  uint16_t tx, ty;
  bool touched = tft.getTouch(&tx, &ty);

  // NEXT short: cycle mode
  if (nextEv == Button::SHORT_PRESS) {
    currentMode = (Mode)((currentMode + 1) % 4);
    const char* labels[] = {
      "BANDSCOPE • SPECTRUM", "BANDSCOPE • OCTAVE",
      "BANDSCOPE • LEVEL",    "BANDSCOPE • EVENTS"
    };
    drawChrome(labels[currentMode]);
  }

  // ENT short: reset peaks
  if (entEv == Button::SHORT_PRESS) {
    // Clear peak holds in audio task
    if (xSemaphoreTake(audioMutex, pdMS_TO_TICKS(20))==pdTRUE) {
      memset(audioData.peakHold, 0, sizeof(audioData.peakHold));
      xSemaphoreGive(audioMutex);
    }
    Serial.println("Peaks reset.");
  }
```

```
  // Touch footer zones
  handleTouch(tx, ty, touched);

  // Get latest audio data
  if (getAudioData(latestAudio) && latestAudio.newFrame) {
    // Render correct mode
    switch (currentMode) {
      case MODE_SPECTRUM: renderSpectrum(latestAudio);   break;
      case MODE_OCTAVE:
        computeOctaveBands();
        renderOctaveBands();
        break;
      case MODE_VOLUME:   renderVolumeMeter(latestAudio); break;
      case MODE_EVENTS:   renderEventLog();               break;
    }
    publishAudio(latestAudio);
    logAudioLevel(latestAudio);
  }

  // Footer uptime update every second
  static unsigned long lastFooter = 0;
  if (now - lastFooter >= 1000) {
    lastFooter = now;
    updateFooter(now, wifiReady(), mqtt.connected());
  }

  if (!otaValid && now > 10000) {
    esp_ota_mark_app_valid_cancel_rollback();
    otaValid = true;
  }
}
```

8 Gain control and Settings.h additions

The software gain multiplier scales the normalised audio samples before FFT computation. Increasing gain makes the spectrum respond to quieter sounds. It is stored in NVS so it persists across reboots.

```
// Settings.h additions for P04
// Add to Settings struct:
  float audioGain = 1.0f;   // 1.0 to 16.0

// Add NVS key:
#define NVS_AUDIO_GAIN "audioGain"   // 9 chars ✓

// Gain adjust via touch zone or NEXT long + ENT:
TouchZone tzGainUp  (84,  282, 72, 36, "GAIN+");
TouchZone tzGainDown(4,   282, 72, 36, "GAIN-");

// In handleTouch():
if (tzGainUp.isTapped(tx, ty, touched)) {
  gainMultiplier = min(gainMultiplier * 2.0f, 16.0f);
  settings.s.audioGain = gainMultiplier;
  settings.save();
  Serial.printf("Gain: x%.0f\n", gainMultiplier);
}
if (tzGainDown.isTapped(tx, ty, touched)) {
  gainMultiplier = max(gainMultiplier / 2.0f, 1.0f);
  settings.s.audioGain = gainMultiplier;
  settings.save();
}

// Gain steps: x1 x2 x4 x8 x16
// x1  = original sensitivity (quiet room only)
// x4  = typical office / classroom
// x8  = outdoor or noisy environment
// x16 = maximum — very sensitive, may clip in loud environments
```

9 Commissioning checklist

☐	Step
☐	Confirm no LED_BLUE usage in firmware. GPIO 32 is exclusively for MEMS_BCLK in BANDSCOPE.
☐	Flash firmware. Confirm Serial shows "Audio on Core 0" and "BANDSCOPE running".
☐	TFT should immediately show spectrum bars responding to ambient sound.
☐	Speak towards the microphone: spectrum bars should rise in the 200-3000Hz region.
☐	Whistle a steady note. Confirm single bar spikes and dominant frequency updates in header.
☐	Clap near the board. Confirm all bars spike simultaneously.
☐	Press NEXT to cycle through SPECTRUM, OCTAVE, LEVEL, EVENTS modes.
☐	In LEVEL mode: speak, clap, whistle. Confirm bar rises smoothly.

☐	In EVENTS mode: clap three times. Confirm events appear with timestamps.
☐	Touch GAIN+ to increase to x4. Confirm spectrum responds to quieter sounds.
☐	After WiFi connects: check MQTT Explorer for e4/audio/dBSPL updating every 5s.
☐	Clap: confirm e4/audio/event publishes immediately.
☐	Open web dashboard. Confirm live level bar visible. After several readings: confirm history chart.
☐	Check SD: confirm AUDIO_LOG.CSV exists and contains dB level rows.
☐	Test OTA: upload new firmware via /update. Confirm audio resumes after reboot.

10 Extending the project

- Add a waterfall display: a scrolling time-frequency plot that shows spectral history as a 2D colour map — each new FFT frame is a row that scrolls downward
- Add pitch detection beyond majorPeak(): implement the HPS (Harmonic Product Spectrum) algorithm for more accurate fundamental frequency detection in speech and music
- Add a musical note display: map dominant frequency to the nearest piano note (A4=440Hz, etc.) and display the note name alongside the frequency
- Add Node-RED noise monitoring dashboard: chart dB SPL over time, alert when sustained levels exceed workplace exposure limits (e.g. >85dB for >2 hours)
- Add DRONELINE integration: use the BANDSCOPE as an acoustic sensor feeding into a wider sensor network via ESP-NOW to the P03 master
- Add a drum beat detector: track the pattern of CLAP events, calculate BPM, and display a live BPM readout on the TFT — relevant to pipe band drumming practice
- Add waveform display mode: raw time-domain oscilloscope view of the audio samples, triggerable on threshold crossing